Solving problems by Searching

Prepared by: Assoc. Prof. Ir. Ts. Dr. Wong Shen Yuong



Why Search?

1. To achieve goals or to maximize our utility we need to predict what the result of our actions in the future will be.

2. There are many sequences of actions, each with their own utility.

3. We want to find, or search for, the best one.

Basic Terminology

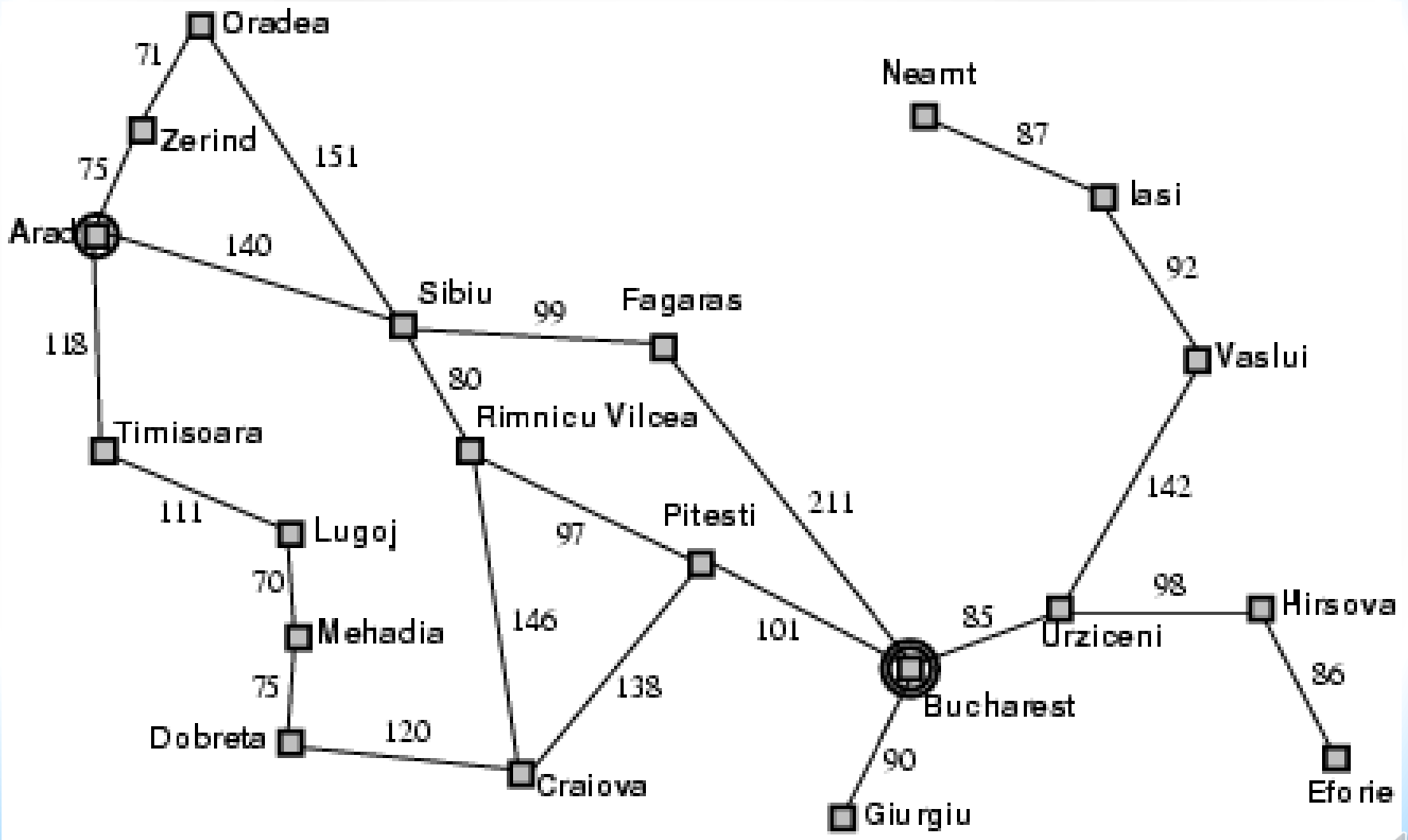
1. **Goal formulation: First step in problem solving. The objectives that the agent is trying to achieve based on current situation and the agent's performance measure.**

2. **Problem formulation: Process of deciding what actions and states to consider, given a goal.**

3. **Search: The process of looking for a sequence of actions that reaches the goal.**

4. **Solution: Result of an action sequence.**

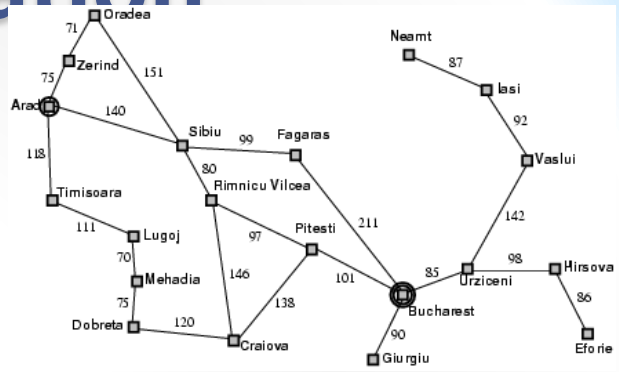
Example: Romania



Example: Romania

1. **On holiday in Romania; currently in Arad.**
2. **Flight leaves tomorrow to Bucharest**
3. **Formulate goal:**
   - be in Bucharest
4. **Formulate problem:**
   - states: various cities
   - actions: drive between cities or choose next city
5. **Find solution:**
   - sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest
   - Cost function: sum of costs of the individual actions along the path.

Problem Formulation



A problem is defined by four items:

initial state e.g., "at Arad“

actions or successor function *S(x)* = set of action–state pairs

- e.g., *S(Arad) = {<Arad → Zerind, Zerind>, … }*

goal test,
e.g., *x* = "at Bucharest”, *Checkmate(x)*

path cost (additive)

- e.g., sum of distances, number of actions executed, etc.
- *c(x,a,y)* is the step cost, assumed to be ≥ 0

A solution is a sequence of actions leading from the initial state to a goal state

Selecting a state space

1. **Real world is absurdly complex**
   → state space must be abstracted for problem solving

2. **Abstraction : Process of removing detail from a representation.**

3. **(Abstract) state ← set of real states**

4. **(Abstract) action ← complex combination of real actions**
   - e.g., "Arad → Zerind" represents a complex set of possible routes, detours, rest stops, etc.

5. **For guaranteed realizability, any real state "in Arad" must get to some real state "in Zerind"**

6. **(Abstract) solution ← set of real paths that are solutions in the real world**

7. **Each abstract action should be "easier" than the original problem**



Environments Types

1. **Static / Dynamic**

   Previous problem was static: no attention to changes in environment

1. **Observable / Partially Observable / Unobservable**

   Previous problem was observable: it knew initial state.

1. **Deterministic / Stochastic**

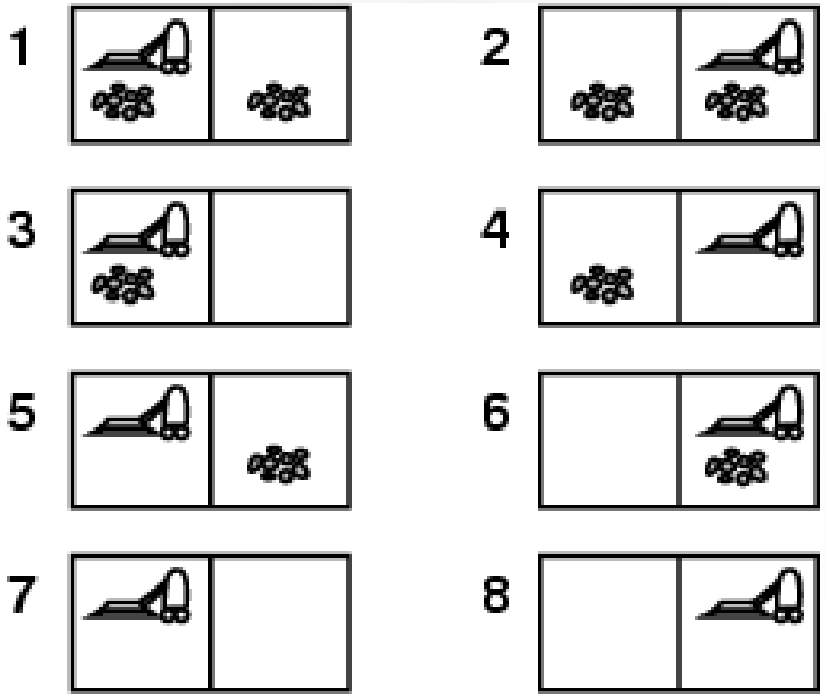
   Previous problem was deterministic: no new percepts were necessary, we can predict the future perfectly given our actions

1. **Discrete / continuous**

   Previous problem was discrete: we can enumerate all possibilities

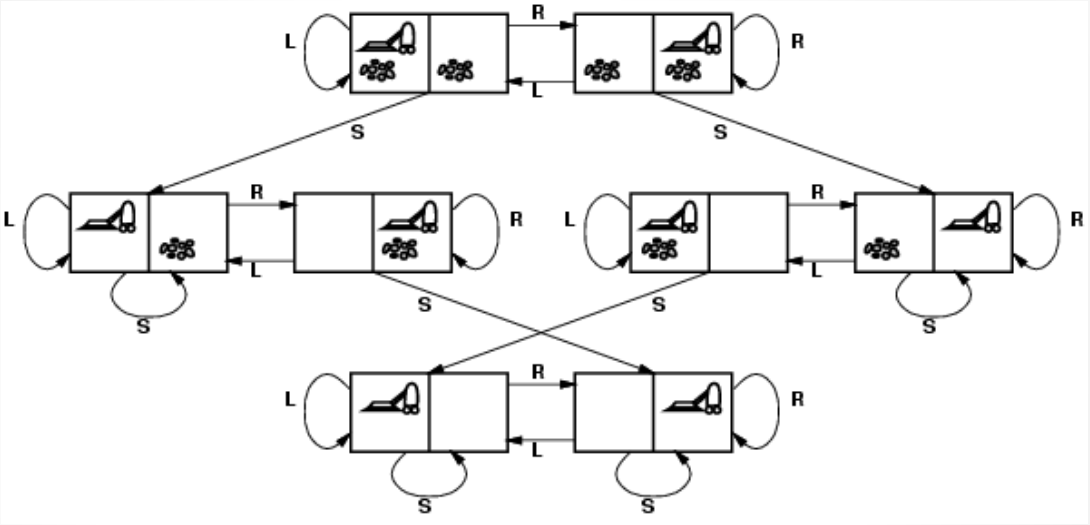
Example: vacuum world

1. **Observable, start in #5.**
   Solution?



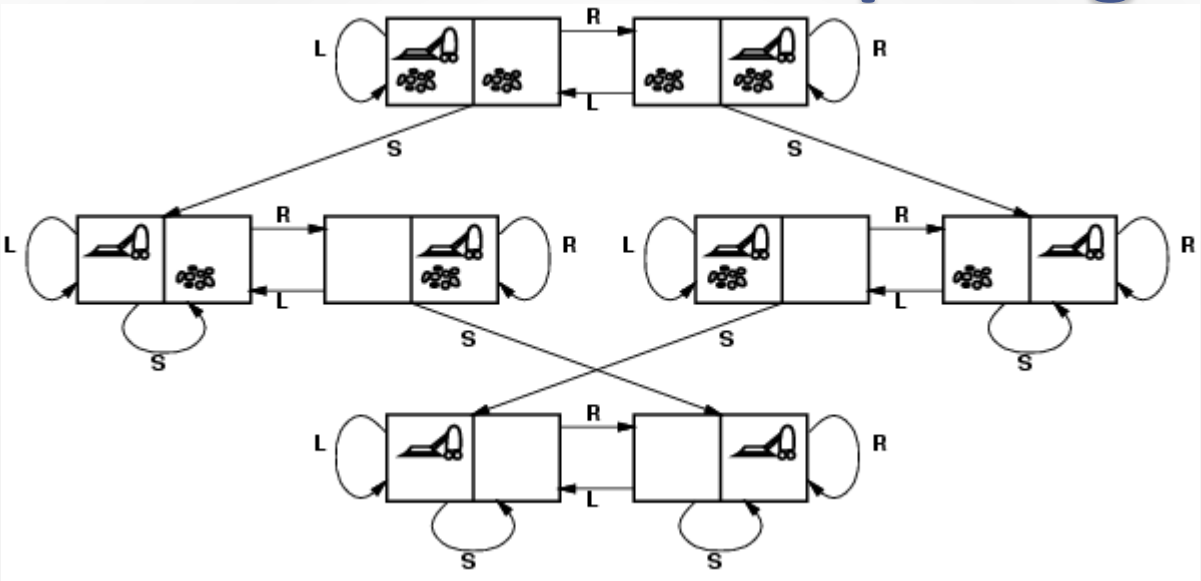
MORE EXAMPLE PROBLEMS

Vacuum world state space graph



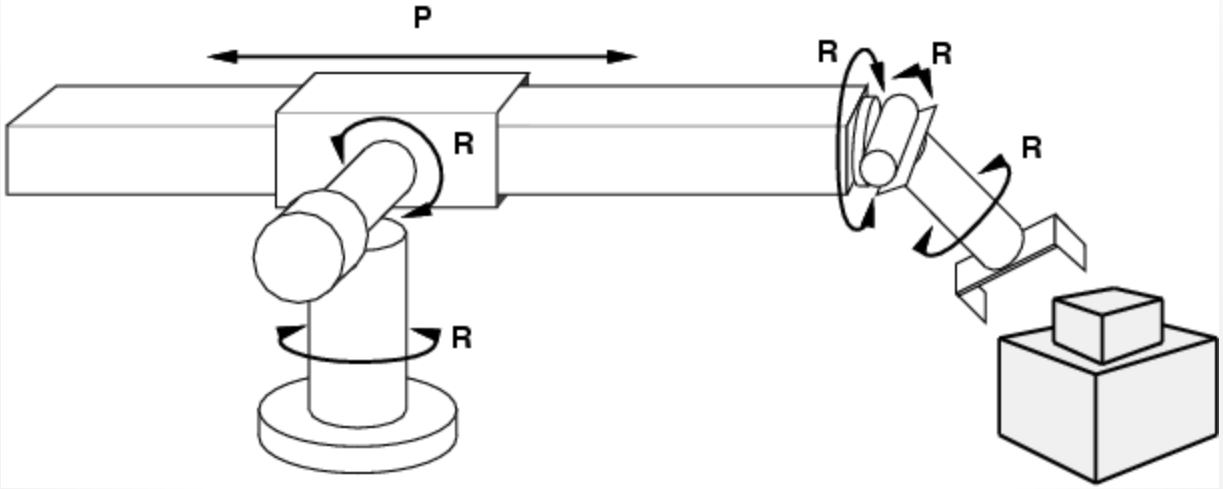
1. states?
2. initial state?
3. actions?
4. goal test?
5. path cost?

Vacuum world state space graph



1. states? discrete: dirt and agent location
2. initial state? Any state can be initial state
3. actions? *Left*, *Right*, *Suck*
4. goal test? no dirt at all locations
5. path cost? 1 per action

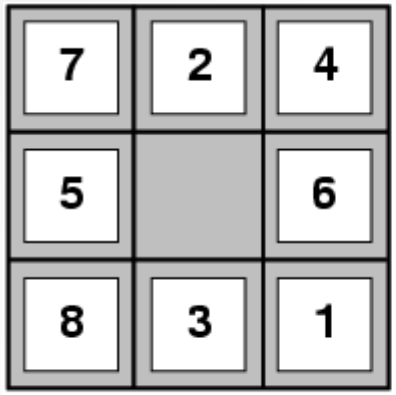
Example: robotic assembly



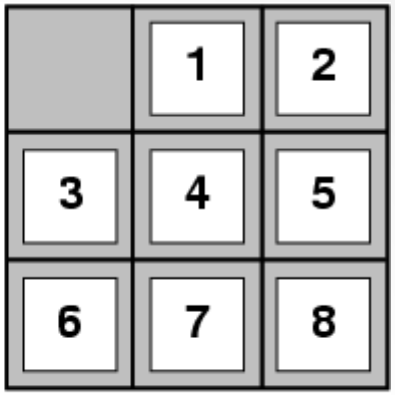
1. states?:
2. initial state?:
3. actions?:
4. goal test?:
5. path cost?:

Try yourselves

Example: The 8-puzzle



Start State



Goal State

1. states?
2. initial state?
3. actions?
4. goal test?
5. path cost?

Try yourselves

Searching for solutions

1. The possible action sequences starting at initial state form a search tree with the initial state at the root; the branches are actions and the nodes correspond to states in the state space of the problem.

2. The process of expanding nodes until either a solution is found or there are no more states to expand – the search strategy.

Search strategies

Strategies are evaluated along the following dimensions:

- completeness: does it always find a solution if one exists?
- time complexity: number of nodes generated
- space complexity: maximum number of nodes in memory
- optimality: does it always find a least-cost solution?